



**Tecnológico
de Monterrey**

TC4031.10

Cómputo en la nube (Gpo 10)

Tarea 8. Usando funciones de Machine Learning en la nube

Jaime Alejandro Mendivil A. A01253316

22 de Marzo del 2026

Introducción

El desarrollo de esta actividad consiste en utilizar los servicios más comunes de plataformas de cómputo en la nube y analizar sus ventajas y desventajas.

Actualmente, las organizaciones requieren procesar y analizar grandes volúmenes de información para realizar tareas como clasificación, regresión y predicción de resultados futuros, con el objetivo de optimizar procesos, reducir errores y mejorar la toma de decisiones.

Plataformas como Microsoft Azure y Google Cloud Platform ofrecen servicios de Machine Learning que permiten implementar estos procesos de forma eficiente, sin la necesidad de infraestructura local avanzada.

En esta práctica, se utilizará Azure Machine Learning (Azure ML) para trabajar con libretas de Python, cargar datos, aplicar técnicas de AutoML y obtener resultados a partir de modelos entrenados.

AutoML en Azure ML

El aprendizaje automático automatizado (AutoML) es una herramienta que permite simplificar el proceso de creación de modelos de Machine Learning. Con Azure ML, AutoML prueba diferentes algoritmos, configuraciones e hiperparámetros para encontrar el modelo con mejor ajuste a los datos y que puedan hacer inferencia y generalizar de forma adecuada.

AutoML automatiza etapas clave como el preprocesamiento de datos, la selección de características, la elección del modelo y la optimización de hiperparámetros. Esto reduce significativamente el tiempo requerido para desarrollar soluciones de Machine Learning, especialmente para usuarios inexpertos.

Además, AutoML se encarga de comparar múltiples modelos mediante métricas de evaluación, así como técnicas de evaluación que aumentan la robustez del modelo sin tender al sobreajuste.

Algunas de las técnicas que implementa automáticamente son:

1. Selección automática de modelos
 - Regresión (Linear Regression, Random Forest, etc.)
 - Clasificación (Logistic Regression, SVM, etc.)
2. Optimización de hiperparámetros
 - Búsqueda aleatoria (random search)
 - Búsqueda inteligente (basada en resultados previos)
3. Feature Engineering
 - Codificación de variables categóricas
 - Normalización / escalado
 - Manejo de valores nulos
4. Feature Selection
 - Precisión
 - Tiempo de entrenamiento
5. Ensemble Learning
 - Voting (promedio de modelos)
 - Stacking (modelo sobre modelos)
6. Manejo automático de datos faltantes

- Media / mediana
- Métodos más avanzados

7. Escalado automático

- Normalización
- Estandarización

8. Evaluación con métricas automáticas

- Clasificación (accuracy, F1-score)
- Regresión (RMSE, MAE)

Implementación mediante API

Empezaremos esta práctica creando un recurso de Machine Learning en el portal de Azure.

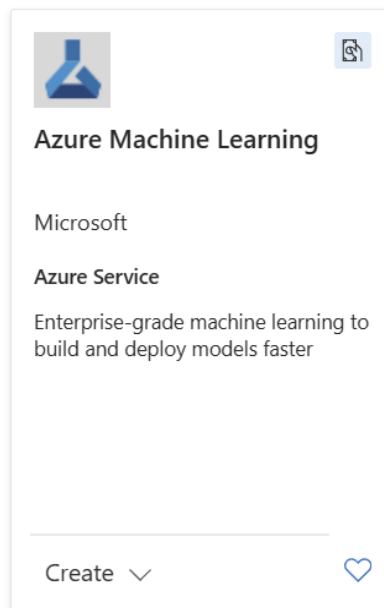


Figura 1. Recurso a crear.

Llenamos los campos con una región en la que estemos habilitados para trabajar.

Home > Marketplace > Azure Machine Learning

Azure Machine Learning

Create a machine learning workspace

Basics

Inbound Access

Outbound Access

Encryption

Identity

Tags

Review + create

Resource details

Every workspace must be assigned to an Azure subscription, which is where billing happens. You use resource groups like folders to organize and manage resources, including the workspace you're about to create.
[Learn more about Azure resource groups](#)

Subscription *

Azure for Students

Resource group *

(New) mna-machinelearning

[Create new](#)

Workspace details

Configure your basic workspace settings like its storage connection, authentication, container, and more. [Learn more](#)

Name *

azureml-mna

Region *

East US

Storage account *

(new) azuremlmna0328902478

[Create new](#)

Key vault *

(new) azuremlmna8432947104

[Create new](#)

Application insights *

(new) azuremlmna9604280424

[Create new](#)

Container registry

None

[Create new](#)

Figura 2. Campos del recurso de AzureML

Y por último en la sección de networking habilitamos el acceso público desde internet. Y con eso creamos nuestro recurso de manera exitosa.

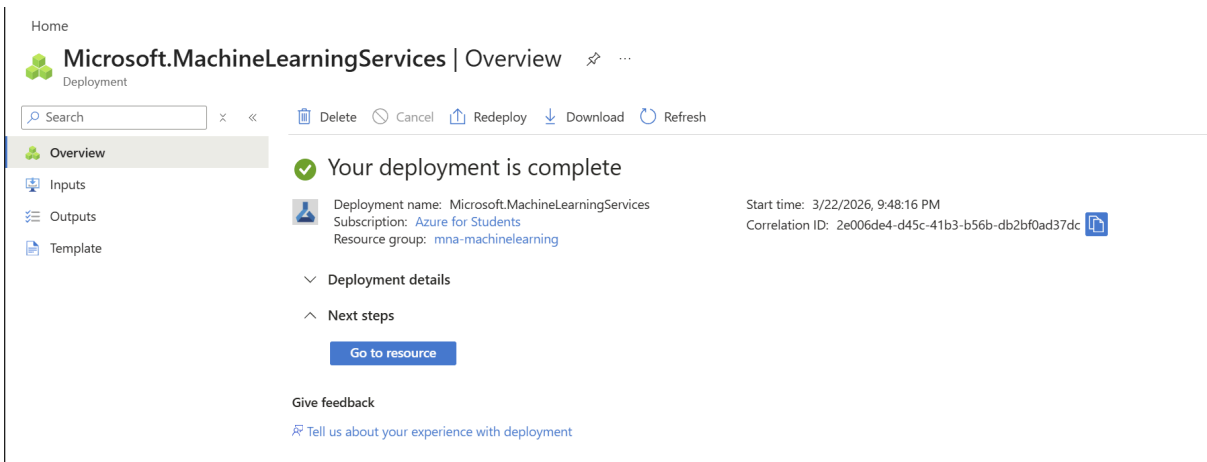


Figura 3. Recurso creado exitosamente.

Una vez dentro del recurso lanzaremos Studio, el IDE en donde manejaremos nuestros datos para crear y evaluar el modelo.

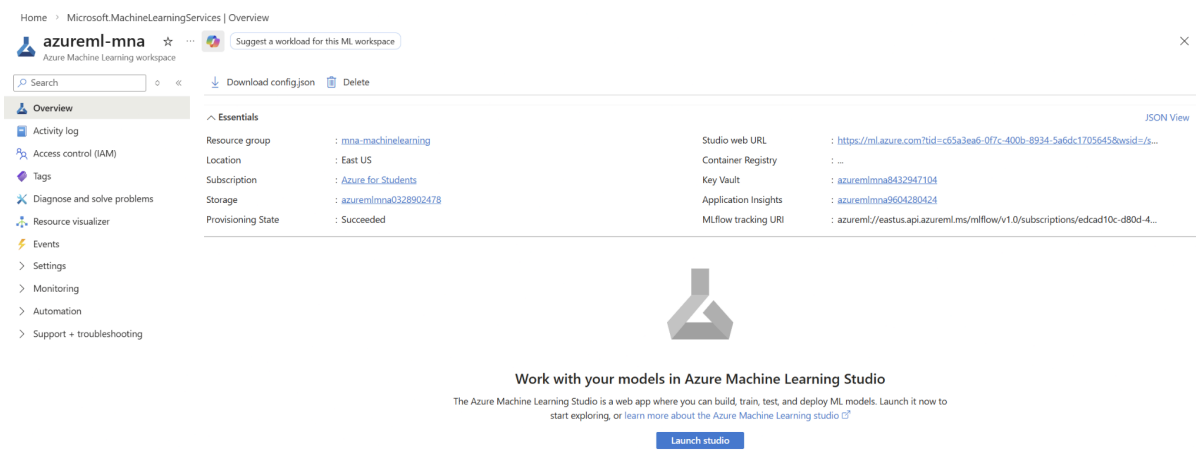


Figura 4. Recurso de Azure Machine Learning Studio

A continuación crearemos un Notebook en donde trabajaremos.

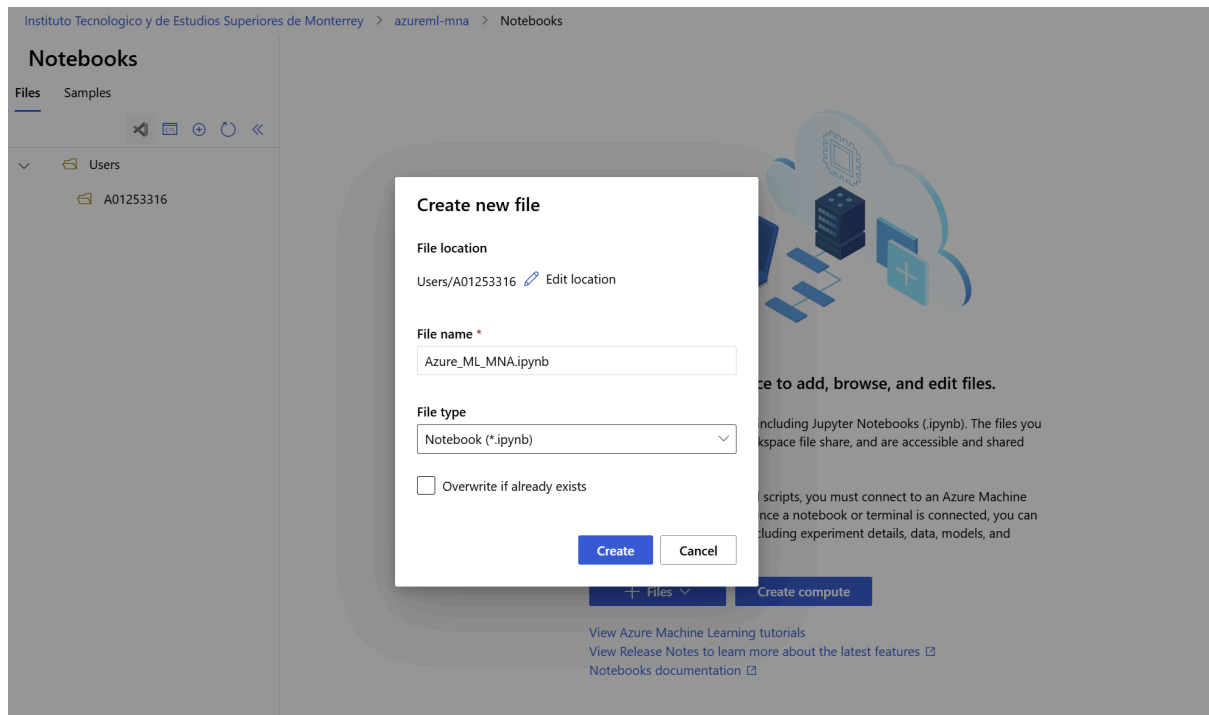


Figura 5. Creación del Notebook de experimentación.

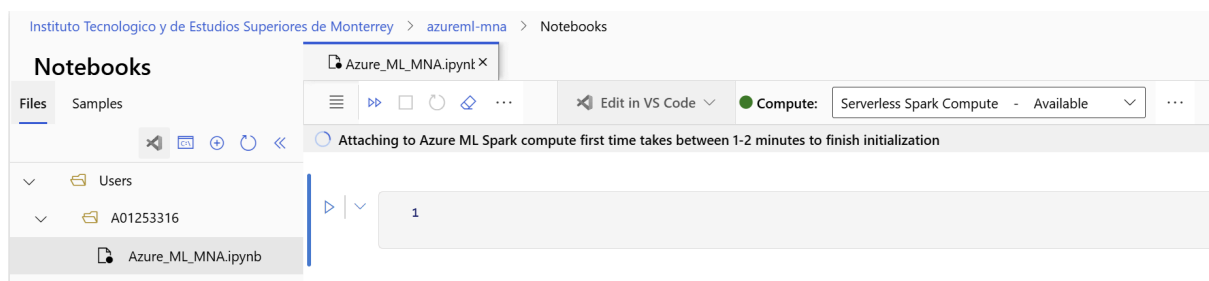


Figura 6. Notebook en Studio

Abrimos el documento y seleccionamos el Kernel/ambiente que ejecutará nuestro código.

Create compute instance

Select the name and virtual machine size you would like to use for your compute instance

Note that a compute instance can not be shared. It can only be used by a single assigned user. By default, it will be assigned to the creator and you can change this to a different user in the Security step.

Compute name *

A01253316-MNA

Virtual machine type

☒ CPU ☐ GPU

Virtual machine size

☒ Select from recommended options ☐ Select from all options

Name ↑	Category	Workload types	Available quota ⓘ	Cost ⓘ
☐ Standard_DS11_v2 2 cores, 14GB RAM, 28GB storage	Memory optimized	Development on Notebooks (or other IDE) and light weight testing	6 cores	\$0.18/hr
☐ Standard_DS3_v2 4 cores, 14GB RAM, 28GB storage	General purpose	Classical ML model training on small datasets	6 cores	\$0.29/hr
☒ Standard_E4ds_v4 4 cores, 32GB RAM, 150GB storage	Memory optimized	Data manipulation and training on medium-sized datasets (1-10GB)	4 cores	\$0.29/hr
☐ Standard_F4s_v2 4 cores, 8GB RAM, 32GB storage	Compute optimiz...	Data manipulation and training on large datasets (> 10 GB)	16 cores	\$0.17/hr

Review + Create Back Next Cancel

Figura 7. Datos de la unidad de cómputo encargada de correr el código.

Una vez instanciada la unidad de cómputo procederemos a subir nuestros datos y empezar a manipularlos.

Carga de los datos

Empezaremos subiendo un archivo de CSV llamado “weatherAUS.csv”, iremos a la sección de Assets y luego a Data, en donde crearemos el archivo subiendo los datos en formato CSV.

The screenshot shows the 'Create data asset' interface. On the left, a vertical sidebar contains two steps: '1 Data type' (highlighted with a blue circle) and '2 Data source'. The main area is titled 'Set the name and type for your data asset'. At the top, there is a yellow warning box with an information icon and text: 'Customers should not include personal data or other sensitive information in fields marked with the [eye icon] because the content in these fields may be logged and shared across Microsoft systems to facilitate operations and troubleshooting. [Learn more](#)'. Below this, there are three fields: 'Name' with the value 'Australian_Rain', 'Description' with the value 'Dataset about amount of rain in Australia', and 'Type' with a dropdown menu showing 'File (uri_file)'.

Figura 8. Creación de Dataset.

Subimos el archivo de manera local.

The screenshot shows the 'Create data asset' interface at the 'Data source' step. The left sidebar now shows '1 Data type' with a green checkmark and '2 Data source' highlighted with a blue circle. The main area is titled 'Choose a source for your data asset' and includes a sub-instruction: 'Choose the data source you want to create your asset from. A data source can be from a local storage location on your computer, from an attached datastore, from Azure storage, or from a publicly available web location.' There are three selectable options, each with a document icon: 'From a URI' (with a description about providing a web URL or URI), 'From Azure storage' (with a description about registered data storage services), and 'From local files' (with a description about uploading files from a local drive). The 'From local files' option is selected, indicated by a blue checkmark in its top right corner.

Figura 9. Tipo de datos de entrada

Seleccionamos donde estarán almacenados.

Create data asset

✓ Data type

✓ Data source

3 Destination storage type

1 File selection

2 Review

Select a datastore

Choose a storage type and a datastore to upload your data to in the next step. You can also create a new datastore for your data first.

Datastore type *

Azure Blob Storage

Create new datastore

Search datastore

Filter

Columns

Name ↓	Storage name	Created on
workspaceblobstore	azuremlmna0328902478	Mar 22, 2026 9:50 PM
workspaceartifactstore	azuremlmna0328902478	Mar 22, 2026 9:50 PM

Figura 10. Destino de los datos.

Y cargamos los datos.

Create data asset

✓ Data type

✓ Data source

✓ Destination storage type

4 File selection

5 Review

Choose a file

Choose a file to upload from your local drive. With the uri_file type, you must upload a single file.

Upload path

azureml://subscriptions/...

Upload file

Overwrite if already exists

Upload list

weatherAUS.csv	✓ 13.44 MB/13.44 MB	...
----------------	---------------------	-----

Figura 11. Subida de Dataset.

Luego agregamos un código de ejemplo en nuestro Notebook para analizar un poco los datos con Pandas.

```

1
2 import pandas as pd
3 from azure.ai.ml import MLClient
4 from azure.identity import DefaultAzureCredential
5
6 ml_client = MLClient.from_config(credential=DefaultAzureCredential())
7 data_asset = ml_client.data.get("Australian_Rain", version="1")
8
9 df = pd.read_csv(data_asset.path)
10 df

```

✓ 20 sec

Figura 12. Código de ejemplo para previsualizar.

	Date	Location	MinTemp	MaxTemp	Rainfall	Evaporation	Sunshine	WindGustDir	WindGustSpeed	WindDir9am	...	Humidity9am	Humidity3pm
0	2008-12-01	Albury	13.4	22.9	0.6	NaN	NaN	W	44.0	W	...	71.0	22.0
1	2008-12-02	Albury	7.4	25.1	0.0	NaN	NaN	WNW	44.0	NNW	...	44.0	25.0
2	2008-12-03	Albury	12.9	25.7	0.0	NaN	NaN	WSW	46.0	W	...	38.0	30.0
3	2008-12-04	Albury	9.2	28.0	0.0	NaN	NaN	NE	24.0	SE	...	45.0	16.0
4	2008-12-05	Albury	17.5	32.3	1.0	NaN	NaN	W	41.0	ENE	...	82.0	33.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...
145455	2017-06-21	Uluru	2.8	23.4	0.0	NaN	NaN	E	31.0	SE	...	51.0	24.0
145456	2017-06-22	Uluru	3.6	25.3	0.0	NaN	NaN	NNW	22.0	SE	...	56.0	21.0
145457	2017-06-23	Uluru	5.4	26.9	0.0	NaN	NaN	N	37.0	SE	...	53.0	24.0
145458	2017-06-24	Uluru	7.8	27.0	0.0	NaN	NaN	SE	28.0	SSE	...	51.0	24.0

Figura 13. Salida del código de ejemplo.

Entrenamiento

Ahora lo que haremos será generar código para empezar a entrenar un modelo a partir de los datos, desde la limpieza de datos, ajuste de datos faltantes, ajuste de hiperparámetros, y métricas de evaluación que aseguren que el modelo no está sobre ajustado y sea capaz de generalizar datos nuevos mediante splits del conjunto de datos.

Para ello haremos uso de la funcionalidad de AutoML, que automáticamente se encarga del entrenamiento sin mucha asistencia por parte de nosotros más que algunos parámetros básicos de entrenamiento.

Empezaremos seleccionando “Automated ML” y seleccionaremos la forma de entrenamiento.

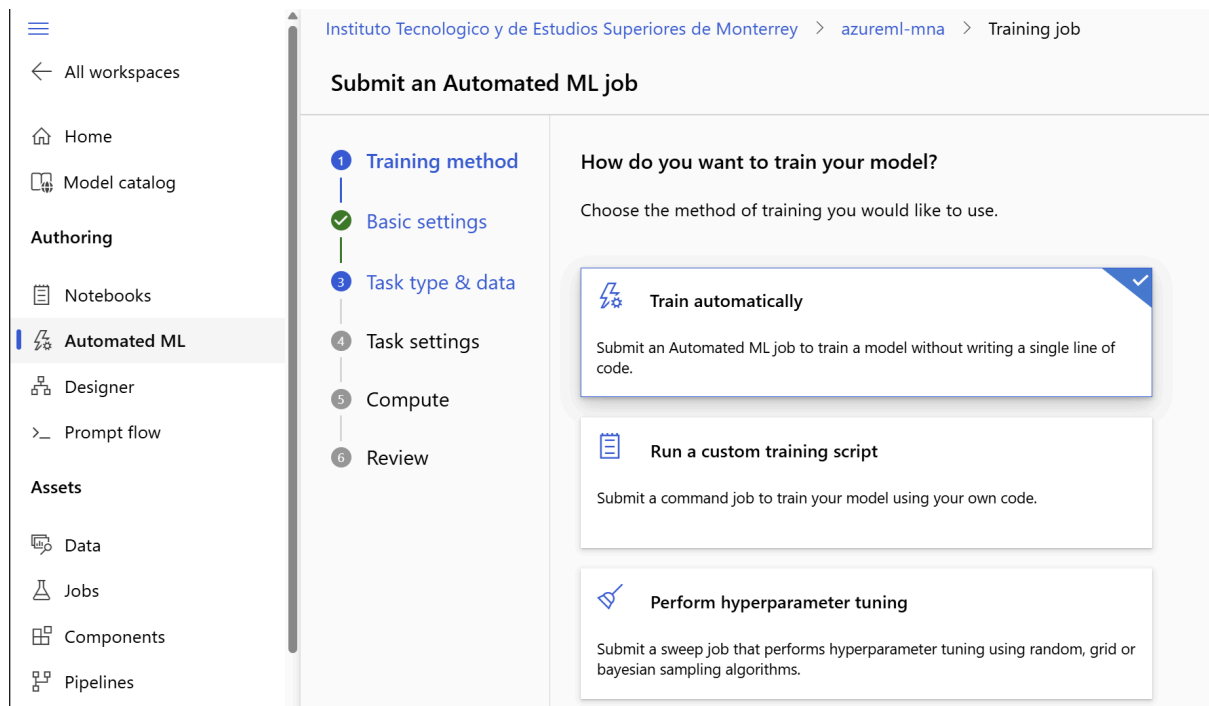


Figura 14. Configuración de Automated ML.

Luego rellenamos las casillas con un poco de información descriptiva del entrenamiento que realizaremos.

Instituto Tecnológico y de Estudios Superiores de Monterrey > azureml-mna > Training job

Submit an Automated ML job

- 1 Training method
- 2 Basic settings
- 3 Task type & data
- 4 Task settings
- 5 Compute
- 6 Review

Basic settings

Let's start with some basic information about your training job.

Job name * ⓘ
Australian_Rain

Experiment name *
☐ Select existing ☒ Create new

New experiment name * ⓘ
test1

Description
Automated Australian rain dataset training.

Tags

Name	:	Value	Add
------	---	-------	-----

Figura 15. Ajustes básicos.

Rellenamos las casillas y seleccionamos el tipo de datos que usaremos, como son datos en formato CSV (comma separated values) por lo que son datos tabulares.

Create data asset

- 1 Data type
- 2 Data source
- 3 Destination storage type
- 4 File or folder selection
- 5 Settings
- 6 Schema
- 7 Review

Set the name and type for your data asset

Name * ⓘ
Australian_Rain_dataset *

Description
Data asset description

Type * ⓘ
Tabular

Figura 16. Tipo de datos con Automated ML.

Luego seleccionaremos el origen del dataset, en nuestro caso sería volver a subir el archivo anterior nuevamente.

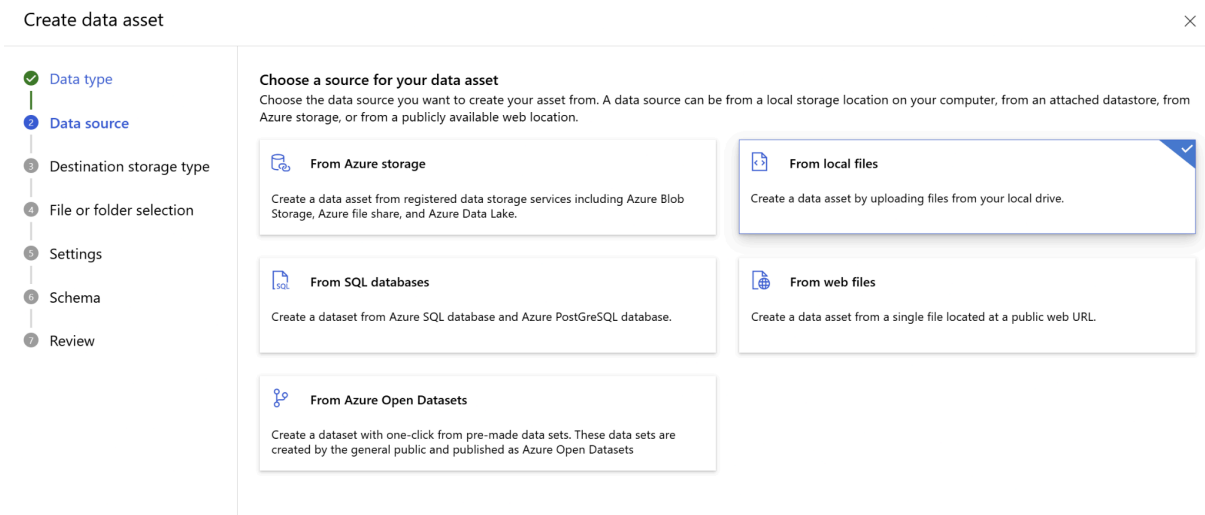


Figura 17. Fuente de datos para AutoML.

Y luego seleccionamos donde hospedamos los datos.

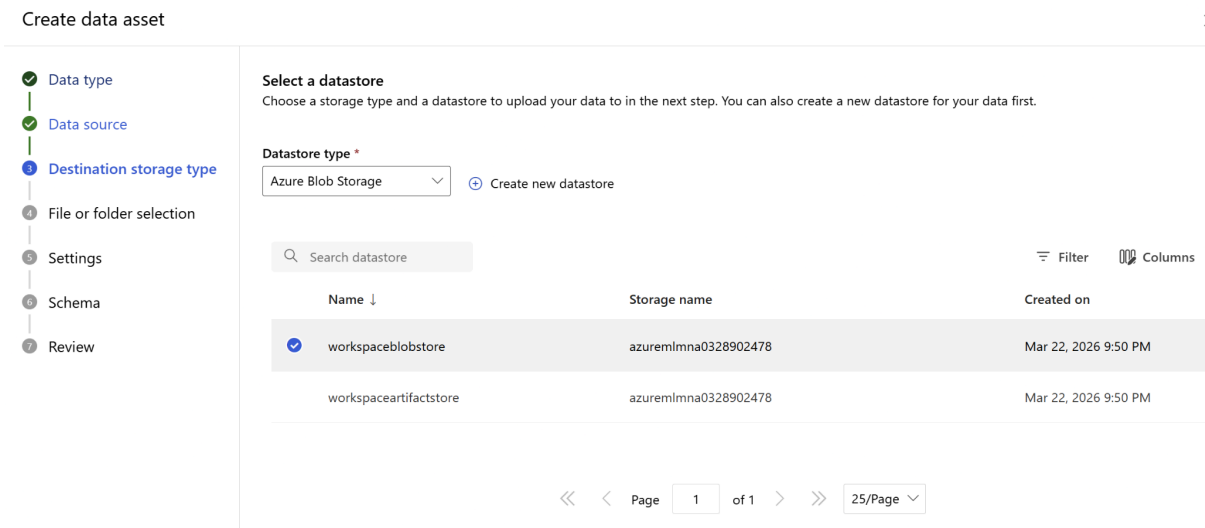


Figura 18. Destino de los datos.

Seguido de esto subiremos el archivo CSV que tenemos descargado.

Create data asset

✓ Data type

✓ Data source

✓ Destination storage type

4 File or folder selection

5 Settings

6 Schema

7 Review

Choose a file or folder

Choose files or folders to upload from your local drive. If you upload multiple folders or files, they will be stored in a containing folder.

Upload path

azureml:/subscriptions/edcad10c-d80d-47bb-bb26-f94b5248348f/resourcegroups/...

Upload files or folder

☐ Overwrite if already exists

Upload list

File Types supported are delimited (i.e. csv, tsv), Parquet, JSON Lines, and plain text.

Figura 19. Subida de datos.

Haremos una interacción rápida para ver qué tan limpio es nuestro dataset, el propósito de este modelo es predecir si el día de mañana lloverá, como se puede observar en la columna de RainTomorrow. Lo mejor sería que estuviera codificada como 0 o 1 pero como tiene datos faltantes y la palabra Yes/No entonces lo dejaremos como un dato de tipo String.

Create data asset

✓ Data type

✓ Data source

✓ Destination storage type

✓ File or folder selection

5 Settings

6 Schema

7 Review

Settings

These settings determine how the data is parsed. The initial settings are automatically detected; you can change them as needed to reparse the data.

File format

Delimited

Delimiter

Comma

Example

Field1,Field2,Field3

Encoding

UTF-8

Column headers

All files have same headers

Skip rows

None

☐ Dataset contains multi-line data

Note: Processing tabular files with multi-line data is slower because multiple CPU cores cannot be used to ingest the data in parallel. Checking this option may result in slower processing times.

Data preview

Date	Location	MinTemp	MaxTe...	Rainfall	Evapor...	Sunshine	WindG...	WindG...	WindDi...	WindDi...	Winds...	Wi
2008-12-...	Albury	13.4	22.9	0.6	NA	NA	W	44	W	WNW	20	24
2008-12-...	Albury	7.4	25.1	0	NA	NA	WNW	44	NNW	WSW	4	22
2008-12-...	Albury	12.9	25.7	0	NA	NA	WSW	46	W	WSW	19	26
2008-12-...	Albury	9.2	28	0	NA	NA	NE	24	SE	E	11	9
2008-12-...	Albury	17.5	32.3	1	NA	NA	W	41	ENE	NW	7	20

Figura 20. Visualización rápida del Dataset.

Aquí editamos de manera específica el formato y propiedades de cada columna del dataset.

Create data asset ×

- ✓ Data type
- ✓ Data source
- ✓ Destination storage type
- ✓ File or folder selection
- ✓ Settings
- 6 Schema**
- 7 Review

Schema

Column types are auto-detected based on the initial subset of the data and can be updated here. Values not aligning with the specified column type will fail conversion and would be either null-filled or replaced with error value. Any conversions preview errors are non-blocking and you can proceed.

Search column name

Include	Column name	Type	Example values	Date format ⓘ	Properties ⓘ
☐	Path	String		Not applicable to s...	Not applicable t...
☒	Date	Date	2008-12-01 00:00:00, 20...	%Y-%m-%d	None
☒	Location	String	Albury, Albury, Albury	Not applicable to s...	Not applicable t...
☒	MinTemp	Decimal (dot '.')	13.4, 7.4, 12.9	Not applicable to s...	Not applicable t...
☒	MaxTemp	Decimal (dot '.')	22.9, 25.1, 25.7	Not applicable to s...	Not applicable t...

Figura 21. Ajustes del Schema de datos.

Luego seleccionamos el tipo de modelo que queremos para nuestro modelo, como la salida debería ser un simple Yes/No, una tarea de regresión logística será suficiente, en este caso clasificación de Yes/No (Añade un umbral para la clasificación de cada etiqueta si la probabilidad excede 0.5).

Instituto Tecnológico y de Estudios Superiores de Monterrey > azureml-mna > Training job

Submit an Automated ML job

- ✓ Training method
- ✓ Basic settings
- ✓ Task type & data
- 4 Task settings**
- 5 Compute
- 6 Review

Task settings

Task type

Classification

Data

Australian_Rain_dataset (View data)

Target column *

RainTomorrow (String) *

Classification settings

☐ Enable deep learning ⓘ

⚙ View additional configuration settings 📄 View featurization settings

Figura 22. Selección de modelo a entrenar y columna de salidas.

Luego seleccionamos los límites de entrenamiento del trabajo, cosas relevantes como el número de épocas, limite de tiempo, nodos máximos (relevante en deep learning)

Instituto Tecnológico y de Estudios Superiores de Monterrey > azureml-mna > Training job

Submit an Automated ML job

- ✓ Training method
- ✓ Basic settings
- ✓ Task type & data
- 4 Task settings
- 5 Compute
- 6 Review

Limits

Max trials ⓘ
50

Max concurrent trials ⓘ
50

Max nodes ⓘ
100

Metric score threshold ⓘ
Enter metric score threshold

Experiment timeout (minutes) ⓘ
60

Iteration timeout (minutes) ⓘ
Enter timeout in minutes

☒ Enable early termination ⓘ

Figura 23. Selección de parámetros del entrenamiento.

También decidiremos varios valores pertinentes para la forma que se valida el modelo. Entre ellos está 'early stopping' que corta el entrenamiento si detecta que el modelo no ha mejorado considerablemente conforme se entrenan más épocas, también como partiremos el dataset para testear su rendimiento y porcentajes de validación.

Instituto Tecnológico y de Estudios Superiores de Monterrey > azureml-mna > Training job

Submit an Automated ML job

- ✓ Training method
- ✓ Basic settings
- ✓ Task type & data
- 4 Task settings
- 5 Compute
- 6 Review

☒ Enable early termination ⓘ

Validate and test

You can choose a validation type and select test data as an optional step.

Validation type ⓘ
Train-validation split

Percentage validation of data * ⓘ
10
Automated ML recommends that between 10 and 30 percent of data is held out for validation

Test data ⓘ
Train-test split

Percentage test of data * ⓘ
20
Automated ML recommends that between 10 and 30 percent of data is held out for test

Figura 24. Ajustes de validación.

Antes de continuar tendremos que decidir en donde ejecutaremos el entrenamiento, como puede ser algo bastante tardado seleccionaremos un cluster de datos que cuente con varios equipos.

Create compute cluster

Virtual Machine | **Advanced Settings**

Configure Settings
Configure compute cluster settings for your selected virtual machine size.

Name	Category	Cores	Available quota	RAM	Storage	Cost/Node
Standard_DS3_v2	General purpose	4	6 cores	14 GB	28 GB	\$0.29/hr

Compute name *
TrainingCluster

Minimum number of nodes *
0

Maximum number of nodes *
1

Idle seconds before scale down *
120

☐ **Enable SSH access**

Figura 25. Creación de cluster de entrenamiento.

Una vez creado seleccionamos el cluster para ejecutar el entrenamiento.

Instituto Tecnológico y de Estudios Superiores de Monterrey > azureml-mna > Training job

Submit an Automated ML job

Training method | **Basic settings** | **Task type & data** | **Task settings** | **Compute** | **Review**

Compute
Select and configure the compute resource for executing your training job.

Select compute type
Compute cluster

Select Azure ML compute cluster *
TrainingCluster, Succeeded, Standard_DS3_v2, 4 vCPUs (cores), 14 GB, 28 GB (storage), \$0.29/hr, No...
[+ New](#)

Figura 26. Selección de cluster.

Y como podemos observar, el trabajo fue creado exitosamente y esperaremos a ver los resultados.

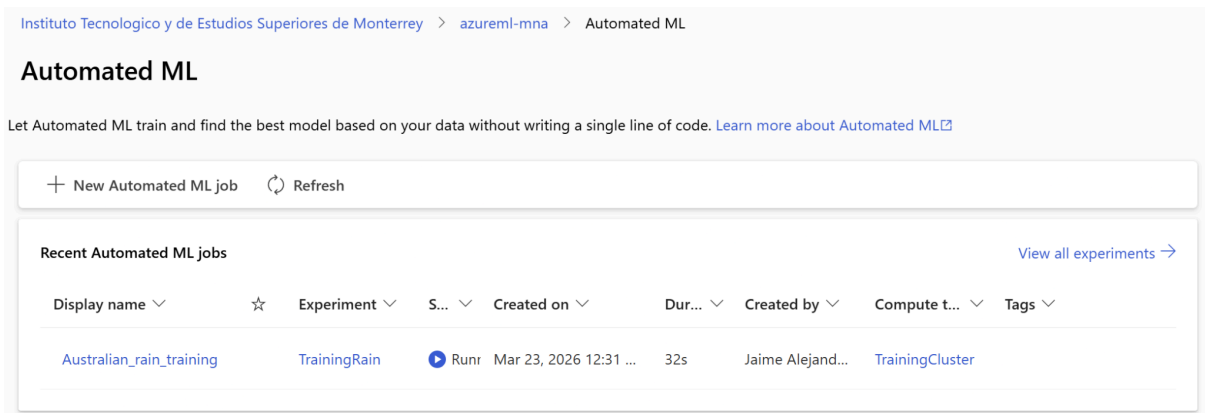


Figura 27. Creación de entrenamiento con AutoML.

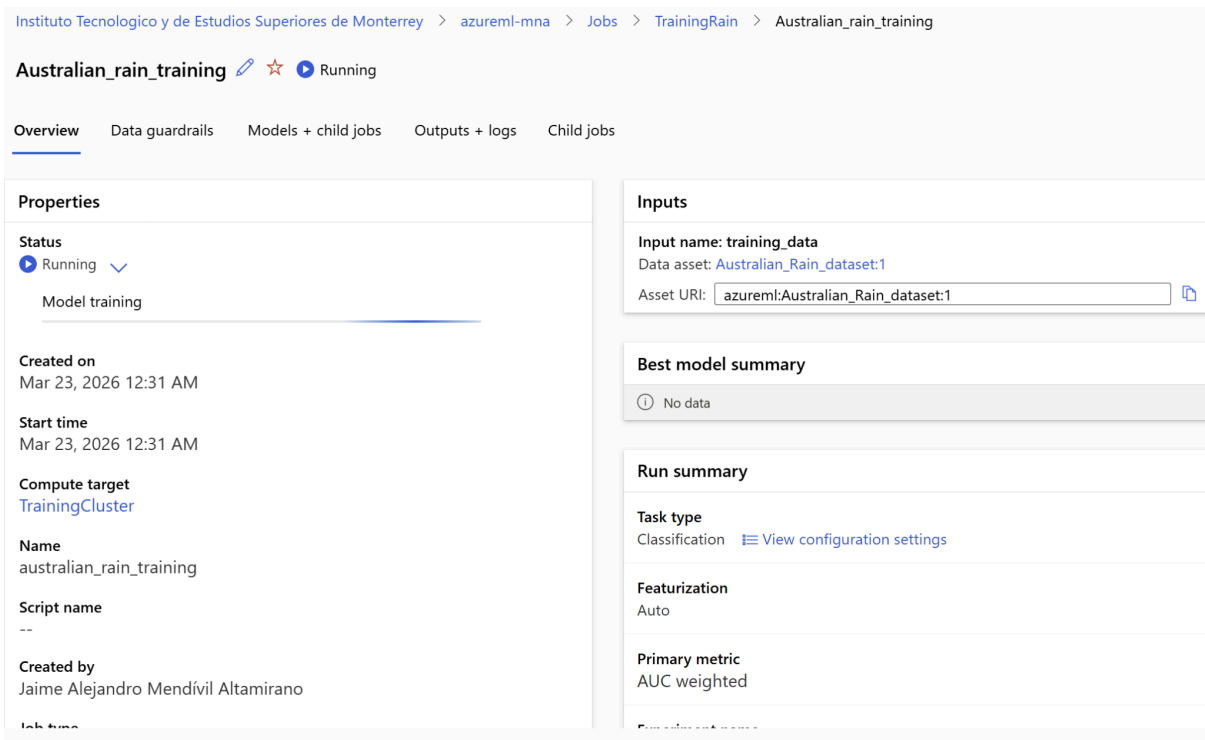


Figura 28. Parámetros generales del trabajo de entrenamiento.

Obtención de los resultados

Ya terminado el tiempo de entrenamiento nos mostrará alertas y los resultados del mejor modelo.

Instituto Tecnológico y de Estudios Superiores de Monterrey > azureml-mna > Jobs > TrainingRain > Australian_rain_training

Australian_rain_training [✎](#) [★](#) [✔](#) Completed

Overview Data guardrails Models + child jobs Outputs + logs Child jobs

🔄 Refresh ⏸ Edit and submit (preview) + Register model ⏹ Cancel 🗑 Delete | 📊 Compare ▾

Properties

Status
✔ Completed ▾

⚠ Warning: No scores improved over last 20 iterations, so experiment stopped early. This early stopping behavior can be disabled by setting enable_early_stopping = False in AutoMLConfig for notebook/python SDK runs.
[See more details](#)

Created on
Mar 23, 2026 12:31 AM

Start time
Mar 23, 2026 12:31 AM

Duration
51m 57.35s

Compute duration
51m 57.35s

Inputs

Input name: training_data
Data asset: [Australian_Rain_dataset:1](#)
Asset URI: [azureml:Australian_Rain_dataset:1](#) [📄](#)

Outputs

Output name: best_model
Model:
[azureml_australian_rain_training_40_output_mlflow_log_model_1653198908:1](#)
Asset URI: [azureml:azureml_australian_rain_training_40_output_mlflow_lo...](#) [📄](#)

Best model summary

Algorithm name
[VotingEnsemble](#)

Figura 29. Ejecución del entrenamiento terminada.

A primera vista podemos observar que el mejor modelo fue un ensamble de votación, que históricamente es uno de los mejores ya que combina varios modelos para decidir los resultados, es usado extensamente en sistemas de recomendaciones de Amazon, Netflix y otras grandes compañías.

Podemos observar también los detalles del ensamble de votación que tienen un AUC Weighted (promedio ponderado del área bajo la curva) de ~0.90 el cual es usado en datasets que cuentan con un imbalance entre el tamaño de ambas clases, el cual podría arrojar valores poco confiables si se usara el de precision para validar el modelo. Cabe recalcar que AutoML se encarga de limpiar los datos antes de hacer algún entrenamiento.

Best model summary	
Algorithm name	VotingEnsemble
Ensemble details	
	View ensemble details
AUC weighted	
0.89449	View all other metrics
Sampling	
100.00 %	?
Registered models	
No registration yet	
Deploy status	
No deployment yet	

Figura 30. Resumen del mejor modelo

También podemos observar que modelos están dentro del ensamble de votación y ver el peso de cada uno para ver cual es el que tiene más relevancia en la predicción de la lluvia.

Ensemble details

Select an ensemble algorithm to see the ensemble weights and hyperparameters.

- ☐ SparseNormalizer, XGBoostClas...
- ☐ StandardScalerWrapper, XGBoo...
- ☐ SparseNormalizer, XGBoostClas...
- ☐ MaxAbsScaler, LinearSVM
- ☐ MaxAbsScaler, LogisticRegressi...
- ☐ StandardScalerWrapper, Rando...
- ☒ MaxAbsScaler, XGBoostClassifier
- ☐ MaxAbsScaler, LightGBM

Ensemble weight: 0.3333333333333333

Data transformation:

```

1  {
2    "spec_class": "preproc",
3    "class_name":
4    "MaxAbsScaler",
5    "module": "sklearn.
6    preprocessing",
7    "param_args": [],
8    "param_kwargs": {},
9    "prepared_kwargs": {}
10 }
```

Training algorithm:

```

1  {
2    "spec_class": "sklearn",
3    "class_name":
4    "XGBoostClassifier",
5    "module": "automl.
6    client.core.common.
7    model_wrappers",
8    "param_args": [],
9    "param_kwargs": {
10     "tree_method":
11     "auto"
12   },
13 }
```

Figura 31. Resultados del ensamble de votación.

Aquí podemos observar que el algoritmo del clasificador XGBoost es el que tiene más peso, algo que tiene mucho sentido ya que es uno de los algoritmos actuales más avanzados y con mejor desempeño en los últimos años, los demás algoritmos tienen de 0.2 a 0.06 de peso en el ensamble de votación. El segundo mejor fue el clasificador al azar de bosques (RFC).

Adicionalmente también podemos observar más métricas del desempeño del modelo dependiendo de que prefieras recall (penaliza falsos negativos) o precisión (penaliza falsos positivos).

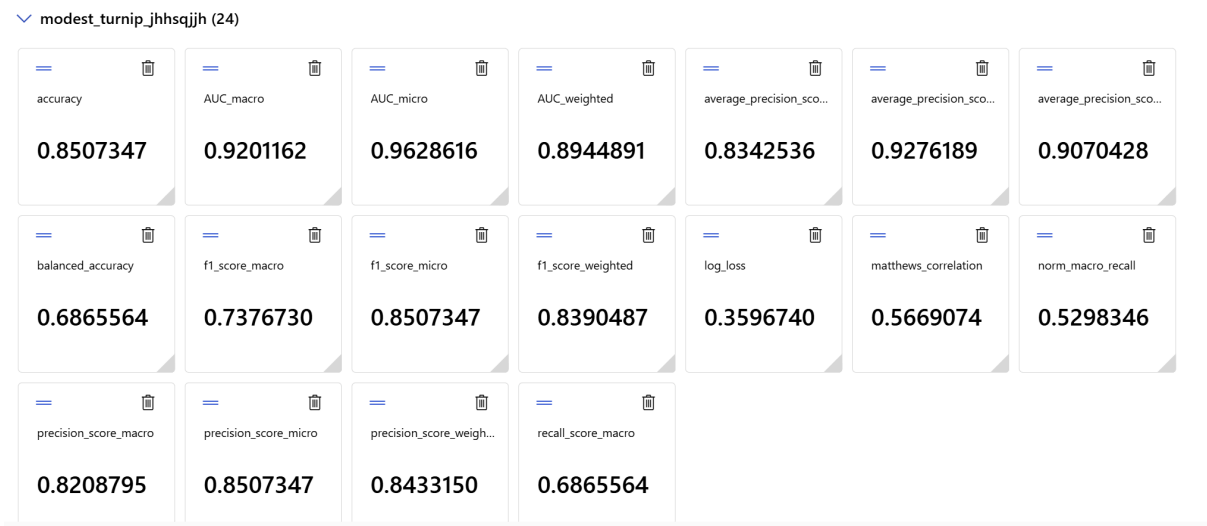


Figura 32. Métricas adicionales.

Así como visualizar distintas gráficas comparativas entre las tasas de aciertos y rechazos.

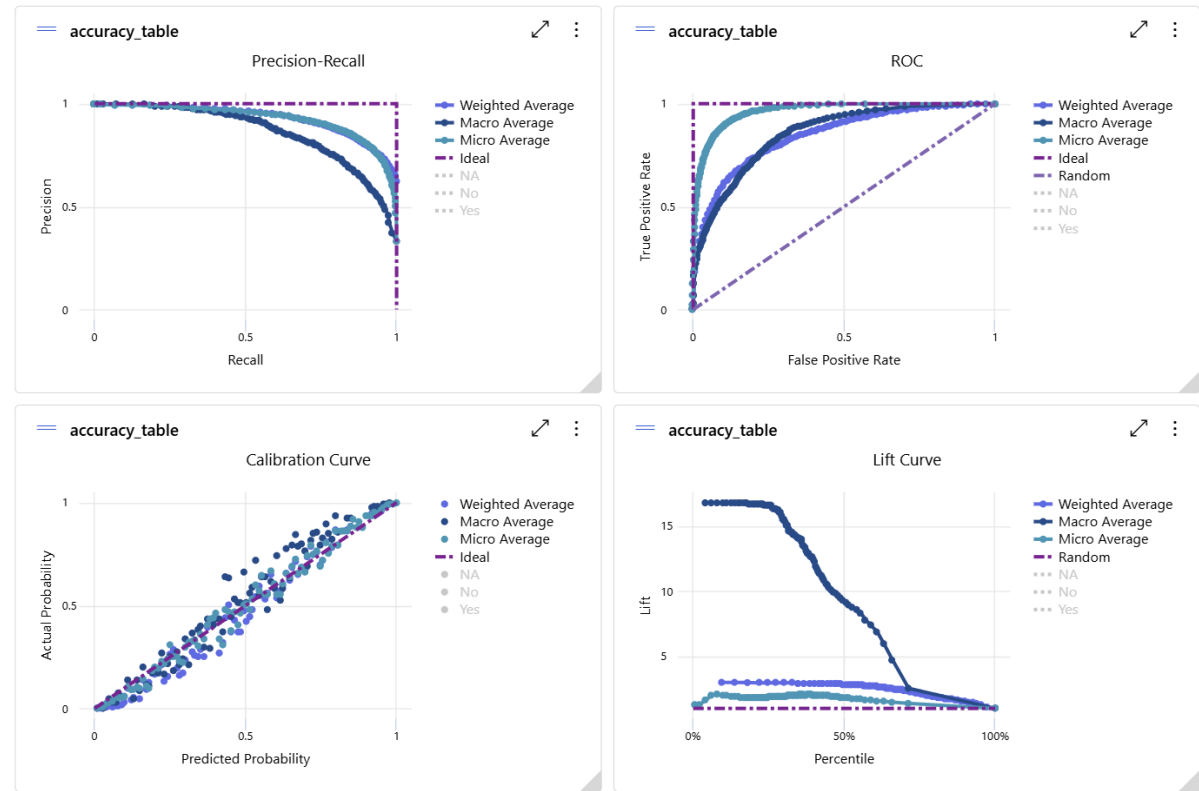


Figura 33. Gráficas comparativas.

También podemos observar las distintas transformaciones que se le aplicaron a los datos antes de ser usados para el entrenamiento.

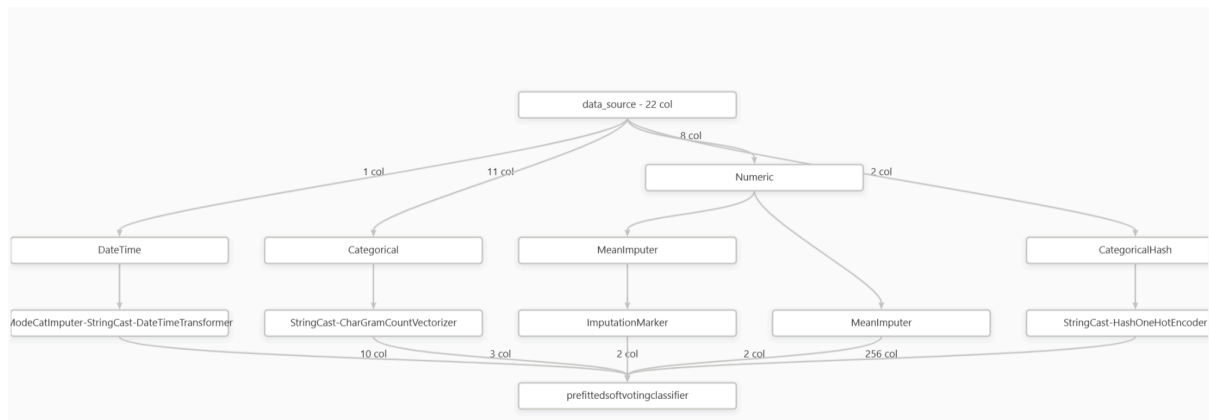


Figura 34. Gráfica de preprocesamiento.

Así como también proporcionar el código aplicado para hacer el entrenamiento.

```

1 import json
2 import os
3
4 from azureml.core.workspace import Workspace
5
6 print("Starting the automl_driver setup...")
7 script_directory = None
8
9 import sys
10 import traceback
11
12 try:
13     from azureml.train.automl._remote_script import setup_wrapper
14     from azureml.train.automl._remote_script import driver_wrapper
15 except Exception as e:
16     print("v2 driver import failed with exception: {}. Falling back to v1 driver.".format(e))
17     traceback.print_exc()
18     import importlib
19     import inspect
20     import logging
21     import os
22     import sys
23     import time
24
25     from automl.client.core.common import utilities
26     from azureml.core.experiment import Experiment
27     from azureml.core.run import Run
28     from azureml.train.automl import automl
29     from azureml.train.automl import fit_pipeline
30     from azureml.train.automl._automl_settings import _AutoMLSettings
31
32 try:
33     from azureml.train.automl._cachestore import _CacheStore
34     from azureml.train.automl._preprocessorcontexts import RawDataContext,
35     TransformedDataContext
36     from azureml.train.automl._transform_data import _transform_data

```

Figura 35. Código del entrenamiento.

Y una vez dado el visto bueno podemos registrar el modelo en distintos formatos para ser visualizado en herramientas como MLFlow así como descargar el modelo y ser desplegado en cualquier aplicación.

Register model from a job output

Figura 36. Registro del modelo obtenido.

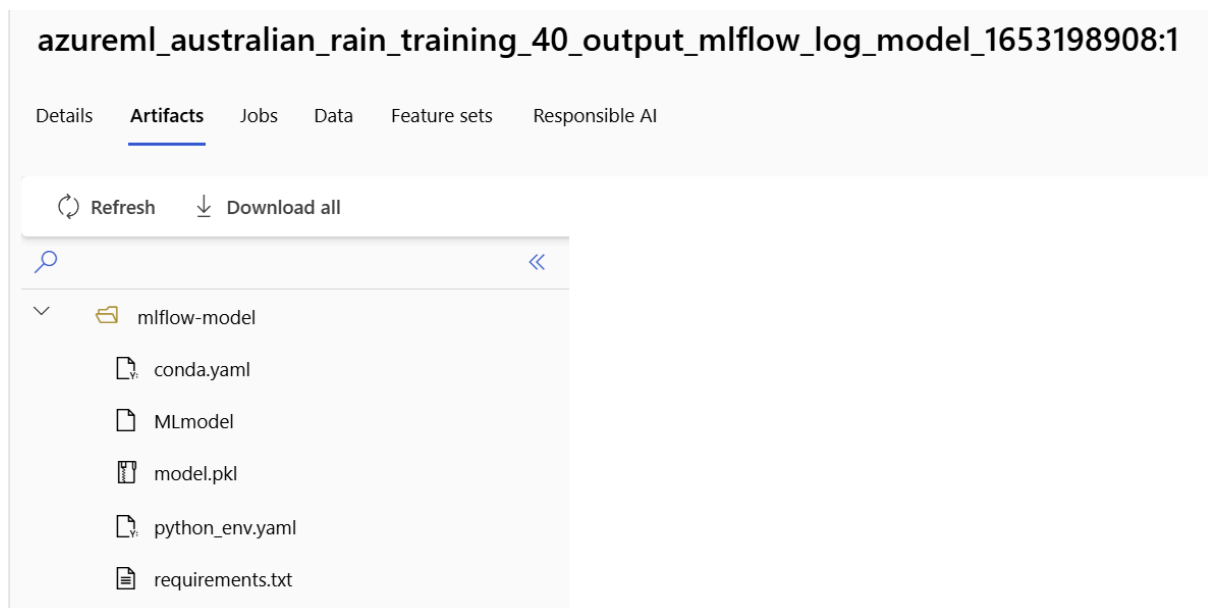


Figura 37. Archivos del modelo

Azure ML y Notebooks

Ahora lo realizaremos de la forma convencional mediante un Notebook. Importamos las librerías y haremos una previsualización rápida del dataset.

```
1 import pandas as pd
2 from azure.ai.ml import MLClient
3 from azure.identity import DefaultAzureCredential
4
5 ml_client = MLClient.from_config(credential=DefaultAzureCredential())
6 data_asset = ml_client.data.get("Australian_Rain", version="1")
7
8 df = pd.read_csv(data_asset.path)
9 df
```

✓ 15 sec

Found the config file in: /config.json
Class DeploymentTemplateOperations: This is an experimental class, and may change at any time. Please see <https://aka.ms/azuremlexperimental> for more information.
Overriding of current TracerProvider is not allowed
Overriding of current LoggerProvider is not allowed
Overriding of current MeterProvider is not allowed
Attempting to instrument while already instrumented
Attempting to instrument while already instrumented
Attempting to instrument while already instrumented
Attempting to instrument while already instrumented
Attempting to instrument while already instrumented
Attempting to instrument while already instrumented
/anaconda/envs/azureml_py310_sdkv2/lib/python3.10/site-packages/mlflow/_init_.py:41: UserWarning: Versions of mlflow (3.8.1) and child packages mlflow-skinny (3.5.0) are different. This may lead to unexpected behavior. Please install the same version of all MLflow packages.
mlflow.mismatch._check_version_mismatch()
Overriding of current TracerProvider is not allowed
Overriding of current LoggerProvider is not allowed
Overriding of current MeterProvider is not allowed
Attempting to instrument while already instrumented
Attempting to instrument while already instrumented
Attempting to instrument while already instrumented
Attempting to instrument while already instrumented
Attempting to instrument while already instrumented

	Date	Location	MinTemp	MaxTemp	Rainfall	Evaporation	Sunshine	WindGustDir	WindGustSpeed	WindDir9am	Humidity9am	Humidity3pm	Pressure9am	Pressure3pm	Cloud9am	Cloud3pm	Temp9am	Temp3pm	RainToday	RainTomorrow
0	2008-12-01	Albury	13.4	22.9	0.6	NaN	NaN	W	44.0	W	71.0	22.0	1007.7	1007.1	8.0	NaN	16.9	21.8	No	No
1	2008-12-02	Albury	7.4	25.1	0.0	NaN	NaN	WNW	44.0	NNW	44.0	25.0	1010.6	1007.8	NaN	NaN	17.2	24.3	No	No
2	2008-12-03	Albury	12.9	25.7	0.0	NaN	NaN	WSW	46.0	W	38.0	30.0	1007.6	1008.7	NaN	2.0	21.0	23.2	No	No
3	2008-12-04	Albury	9.2	28.0	0.0	NaN	NaN	NE	24.0	SE	45.0	16.0	1017.6	1012.8	NaN	NaN	18.1	26.5	No	No
4	2008-12-05	Albury	17.5	32.3	1.0	NaN	NaN	W	41.0	ENE	82.0	33.0	1010.8	1006.0	7.0	8.0	17.8	29.7	No	No
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
145455	2017-06-21	Ulluru	2.8	23.4	0.0	NaN	NaN	E	31.0	SE	51.0	24.0	1024.6	1020.3	NaN	NaN	10.1	22.4	No	No
145456	2017-06-22	Ulluru	3.6	25.3	0.0	NaN	NaN	NNW	22.0	SE	56.0	21.0	1023.5	1019.1	NaN	NaN	10.9	24.5	No	No
145457	2017-06-23	Ulluru	5.4	26.9	0.0	NaN	NaN	N	37.0	SE	53.0	24.0	1021.0	1016.8	NaN	NaN	12.5	26.1	No	No
145458	2017-06-24	Ulluru	7.8	27.0	0.0	NaN	NaN	SE	28.0	SSE	51.0	24.0	1019.4	1018.5	3.0	2.0	15.1	26.0	No	No
145459	2017-06-25	Ulluru	14.9	NaN	0.0	NaN	NaN	NaN	NaN	ESE	62.0	36.0	1020.2	1017.9	8.0	8.0	15.0	20.9	No	NaN

Figura 38. Carga de librerías y dataset.

Ahora cargaremos nuestro dataset y empezaremos a limpiarlo así como importar los modelos con los que trabajaremos.

```
1 import pandas as pd
2 import numpy as np
3 import os
4 import joblib
5
6 from azure.ai.ml import MLClient
7 from azure.identity import DefaultAzureCredential
8
9 from sklearn.model_selection import train_test_split, StratifiedFold, RandomizedSearchCV
10 from sklearn.compose import ColumnTransformer
11 from sklearn.pipeline import Pipeline
12 from sklearn.impute import SimpleImputer
13 from sklearn.preprocessing import OneHotEncoder
14 from sklearn.ensemble import RandomForestClassifier
15 from sklearn.metrics import (
16     accuracy_score,
17     precision_score,
18     recall_score,
19     f1_score,
20     roc_auc_score,
21     confusion_matrix,
22     classification_report
23 )
24
25 # =====
26 # 1. Cargan dataset desde Azure ML
27 # =====
28 ml_client = MLClient.from_config(credential=DefaultAzureCredential())
29 data_asset = ml_client.data.get("Australian_Rain", version="1")
30
31 df = pd.read_csv(data_asset.path)
32
33 print("Dimensiones del dataset:", df.shape)
34 display(df.head())
35 display(df.info())
36
```

✓ 4 sec

Figura 39. Carga del dataset.

Obteniendo así los siguientes resultados. En esta parte veremos la cantidad de entradas por columna así como el tipo de dato de cada una.

	Date	Location	MinTemp	MaxTemp	Rainfall	Evaporation	Sunshine	WindGustDir	WindGustSpeed	WindDir9am	...	Humidity9am	Humidity3pm	Pressure9am	Pressure3pm	Cloud9am	Cloud3pm	Temp9am	Temp3pm	RainToday	RainTomorrow
0	2008-12-01	Albury	13.4	22.9	0.6	NaN	NaN	W	44.0	W	...	71.0	22.0	1007.7	1007.1	8.0	NaN	16.9	21.8	No	No
1	2008-12-02	Albury	7.4	25.1	0.0	NaN	NaN	WNW	44.0	NNW	...	44.0	25.0	1010.6	1007.8	NaN	NaN	17.2	24.3	No	No
2	2008-12-03	Albury	12.9	25.7	0.0	NaN	NaN	WSW	46.0	W	...	38.0	30.0	1007.6	1008.7	NaN	2.0	21.0	23.2	No	No
3	2008-12-04	Albury	9.2	28.0	0.0	NaN	NaN	NE	24.0	SE	...	45.0	16.0	1017.6	1012.8	NaN	NaN	18.1	26.5	No	No
4	2008-12-05	Albury	17.5	32.3	1.0	NaN	NaN	W	41.0	ENE	...	82.0	33.0	1010.8	1006.0	7.0	8.0	17.8	29.7	No	No

5 rows x 23 columns

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 145460 entries, 0 to 145459
Data columns (total 23 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   Date                  145460 non-null object
1   Location              145460 non-null object
2   MinTemp               143975 non-null float64
3   MaxTemp               144199 non-null float64
4   Rainfall              142199 non-null float64
5   Evaporation           82670 non-null float64
6   Sunshine              75625 non-null float64
7   WindGustDir           135134 non-null object
8   WindGustSpeed         135197 non-null float64
9   WindDir9am            134894 non-null object
10  WindDir3pm            141232 non-null object
11  WindSpeed9am          143693 non-null float64
12  WindSpeed3pm          142398 non-null float64
13  Humidity9am           142806 non-null float64
14  Humidity3pm           140953 non-null float64
15  Pressure9am           130395 non-null float64
16  Pressure3pm           130432 non-null float64
17  Cloud9am              89572 non-null float64
18  Cloud3pm              86102 non-null float64
19  Temp9am               143693 non-null float64
20  Temp3pm               141851 non-null float64
21  RainToday             142199 non-null object
22  RainTomorrow          142193 non-null object
dtypes: float64(16), object(7)
memory usage: 25.5+ MB
None

```

Figura 40. Tipos de datos del dataset.

Después buscaremos la cantidad de datos faltantes por columna.

```

1
2 # =====
3 # 2. Limpieza inicial
4 # =====
5 df = df.drop_duplicates().copy()
6
7 # Estandarizar posibles faltantes
8 missing_tokens = ["", " ", "NA", "N/A", "na", "null", "NULL", "?", "None"]
9 df = df.replace(missing_tokens, np.nan)
10
11 # Limpiar nombres de columnas
12 df.columns = df.columns.str.strip()
13
14 print("\nValores faltantes por columna:")
15 display(df.isna().sum().sort_values(ascending=False))
16
3] ✓ <1 sec

```

```

Valores faltantes por columna:
Sunshine      69835
Evaporation   62790
Cloud3pm      59358
Cloud9am      55888
Pressure9am   15065
Pressure3pm   15028
WindDir9am    10566
WindGustDir   10326
WindGustSpeed 10263
Humidity3pm    4507
WindDir3pm    4228
Temp3pm       3609
RainTomorrow  3267
Rainfall      3261
RainToday     3261
WindSpeed3pm  3062
Humidity9am   2654
Temp9am       1767
WindSpeed9am  1767
MinTemp       1485
MaxTemp       1261
Location      0
Date          0
dtype: int64

```

Figura 41. Datos faltantes por columna.

De esta forma vemos cuáles datos hay que eliminar, cuales hay que rellenar y cuales hay que conservar. También tenemos que separar los distintos valores en la columna a predecir para ver si hay algún desbalance de clases que hagan que el modelo esté sesgado a cualquiera de las clases.

```

1
2 # =====
3 # 3. Preparación del target
4 # =====
5 target_col = "RainTomorrow"
6
7 # Eliminar filas donde falte la variable objetivo
8 df = df.dropna(subset=[target_col]).copy()
9
10 # Convertir target a binario
11 df[target_col] = df[target_col].astype(str).str.strip().str.lower()
12 df[target_col] = df[target_col].map({"yes": 1, "no": 0})
13
14 # Eliminar filas que no hayan podido mapearse
15 df = df.dropna(subset=[target_col]).copy()
16 df[target_col] = df[target_col].astype(int)
17
18 # También convertir RainToday si existe
19 if "RainToday" in df.columns:
20     df["RainToday"] = df["RainToday"].astype(str).str.strip().str.lower()
21     df["RainToday"] = df["RainToday"].map({"yes": 1, "no": 0})
22
23 print("\nDistribución de clases:")
24 print(df[target_col].value_counts())
25 print(df[target_col].value_counts(normalize=True))
26
4] ✓ <1 sec

```

Distribución de clases:

```

0    110316
1     31877
Name: RainTomorrow, dtype: int64
0    0.775819
1    0.224181
Name: RainTomorrow, dtype: float64

```

Figura 42. Distribución de clases.

Como podemos observar hay más entradas para una de las clases por lo que métricas como “accuracy” aunque muestren un alto valor, pueden significar que erran en más del 90% de la clase más pequeña, algo que es perjudicial para el modelo que entrenemos, se podría hacer “data augmentation” para equilibrar ambas clases pero por cuestiones de facilidad simplemente usaremos una métrica dedicada para casos en donde se presenten clases no balanceadas.

También convertiremos ciertas columnas como la fecha a otras columnas más representativas de la probabilidad de lluvia, en este caso los meses y ciertos años.

```

1
2 # =====
3 # 4. Ingeniería básica de características
4 # =====
5 # Convertir Date en variables útiles
6 if "Date" in df.columns:
7     df["Date"] = pd.to_datetime(df["Date"], errors="coerce")
8     df["Year"] = df["Date"].dt.year
9     df["Month"] = df["Date"].dt.month
10    df["Day"] = df["Date"].dt.day
11    df = df.drop(columns=["Date"])
12
✓ <1 sec

```

Figura 43. Feature engineering.

Adicionalmente separamos columnas categóricas y numéricas. También empezaremos a separar el dataset en variables de entrada y la variable a predecir.

```

1
2 # =====
3 # 5. Separar X e y
4 # =====
5 X = df.drop(columns=[target_col])
6 y = df[target_col]
7
8 # Identificar columnas numéricas y categóricas
9 numeric_features = X.select_dtypes(include=["int64", "float64"]).columns.tolist()
10 categorical_features = X.select_dtypes(include=["object"]).columns.tolist()
11
12 print("\nColumnas numéricas:", numeric_features)
13 print("\nColumnas categóricas:", categorical_features)
14
✓ <1 sec

```

Columnas numéricas: ['MinTemp', 'MaxTemp', 'Rainfall', 'Evaporation', 'Sunshine', 'WindGustSpeed', 'WindSpeed9am', 'WindSpeed3pm', 'Humidity9am', 'Humidity3pm', 'Pressure9am', 'Pressure3pm', 'Cloud9am', 'Cloud3pm', 'Temp9am', 'Temp3pm', 'RainToday', 'Year', 'Month', 'Day']

Columnas categóricas: ['Location', 'WindGustDir', 'WindDir9am', 'WindDir3pm']

Figura 44. Separación de variables categóricas y numéricas.

Después separaremos nuestro dataset en distintas partes para entrenamiento, validación y sintonización de hiperparametros así como ver la cantidad por set.

```
1
2 # =====
3 # 6. Dividir datos en train, validation y test
4 # =====
5 X_trainval, X_test, y_trainval, y_test = train_test_split(
6     X, y,
7     test_size=0.20,
8     stratify=y,
9     random_state=42
10 )
11
12 X_train, X_val, y_train, y_val = train_test_split(
13     X_trainval, y_trainval,
14     test_size=0.25, # 0.25 de 0.80 = 0.20 total
15     stratify=y_trainval,
16     random_state=42
17 )
18
19 print("\nShapes:")
20 print("Train:", X_train.shape, y_train.shape)
21 print("Val:", X_val.shape, y_val.shape)
22 print("Test:", X_test.shape, y_test.shape)
23
```

✓ <1 sec

Shapes:
Train: (85315, 24) (85315,)
Val: (28439, 24) (28439,)
Test: (28439, 24) (28439,)

Figura 45. Split de datos

También empezamos a crear nuestro pipeline de limpieza de datos que organizará, separará e imputará cualquier dato faltante y ruidoso para que sea de utilidad para el entrenamiento.

```
1
2 # =====
3 # 7. Preprocesamiento
4 # =====
5 numeric_transformer = Pipeline(steps=[
6     ("imputer", SimpleImputer(strategy="median"))
7 ])
8
9 categorical_transformer = Pipeline(steps=[
10     ("imputer", SimpleImputer(strategy="most_frequent")),
11     ("onehot", OneHotEncoder(handle_unknown="ignore"))
12 ])
13
14 preprocessor = ColumnTransformer(
15     transformers=[
16         ("num", numeric_transformer, numeric_features),
17         ("cat", categorical_transformer, categorical_features)
18     ]
19 )
20
```

✓ <1 sec

Figura 45. Pipeline de preprocesamiento.

Ahora inicializamos un modelo para que limpie y ajuste el modelo al dataset de entrenamiento.

```
1
2 # =====
3 # 8. Pipeline con modelo
4 # =====
5 rf_model = RandomForestClassifier(
6     random_state=42,
7     n_jobs=-1,
8     class_weight="balanced"
9 )
10
11 pipeline = Pipeline(steps=[
12     ("preprocessor", preprocessor),
13     ("model", rf_model)
14 ])
15
✓ <1 sec
```

Figura 46. Inicializacion de modelo RF

Ahora lo que haremos será definir los hiperparametros que deseamos optimizar así como aplicar validación cruzada para que al ajustar el modelo entre distintas partes del dataset, este sea capaz de generalizar correctamente y reaccionar a datos nuevos sin que este memorizando el set de entrenamiento y sea incapaz de predecir nuevos datos.

```
1
2 # =====
3 # 9. Ajuste de hiperparámetros con validación cruzada
4 # =====
5 param_distributions = {
6     "model__n_estimators": [100, 200, 300],
7     "model__max_depth": [None, 10, 20, 30],
8     "model__min_samples_split": [2, 5, 10],
9     "model__min_samples_leaf": [1, 2, 4],
10    "model__max_features": ["sqrt", "log2", None]
11 }
12
13 cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
14
15 search = RandomizedSearchCV(
16     estimator=pipeline,
17     param_distributions=param_distributions,
18     n_iter=15,
19     scoring="f1",
20     cv=cv,
21     verbose=2,
22     random_state=42,
23     n_jobs=-1,
24     return_train_score=True
25 )
26
27 search.fit(X_train, y_train)
28
29 print("\nMejores hiperparámetros:")
30 print(search.best_params_)
31 print("\nMejor score de validación cruzada:", search.best_score_)
32
33 best_model = search.best_estimator_
34
✓ 1 h 51 min
```

Fitting 5 folds for each of 15 candidates, totalling 75 fits
[CV] END model__max_depth=10, model__max_features=log2, model__min_samples_leaf=4, model__min_samples_split=10, model__n_estimators=100; total time= 16.7s
[CV] END model__max_depth=10, model__max_features=log2, model__min_samples_leaf=4, model__min_samples_split=10, model__n_estimators=100; total time= 16.0s
[CV] END model__max_depth=10, model__max_features=log2, model__min_samples_leaf=1, model__min_samples_split=2, model__n_estimators=100; total time= 15.5s

Figura 47. Entrenamiento de modelo y validación cruzada.

Y como podemos observar nos regresa los mejores hiperparametros para el modelo de “Random Forests”.

```
Mejores hiperparámetros:  
{'model__n_estimators': 300, 'model__min_samples_split': 5, 'model__min_samples_leaf': 4, 'model__max_features': 'sqrt', 'model__max_depth': 30}  
  
Mejor score de validación cruzada: 0.6541041842312394
```

Figura 48. Mejores hiperparametros para RF.

Ahora validamos el modelo con el dataset de validación para ver que tan bien se desempeña en datos no observados.

```
1  
2 # =====  
3 # 10. Función de evaluación  
4 # =====  
5 def evaluate_model(model, X_data, y_true, dataset_name="dataset"):  
6     y_pred = model.predict(X_data)  
7     y_proba = model.predict_proba(X_data)[: , 1]  
8  
9     metrics = {  
10         "dataset": dataset_name,  
11         "accuracy": accuracy_score(y_true, y_pred),  
12         "precision": precision_score(y_true, y_pred, zero_division=0),  
13         "recall": recall_score(y_true, y_pred, zero_division=0),  
14         "f1": f1_score(y_true, y_pred, zero_division=0),  
15         "roc_auc": roc_auc_score(y_true, y_proba)  
16     }  
17  
18     print(f"\n==== {dataset_name.upper()} =====")  
19     for k, v in metrics.items():  
20         if k != "dataset":  
21             print(f"{k}: {v:.4f}")  
22  
23     print("\nMatriz de confusión:")  
24     print(confusion_matrix(y_true, y_pred))  
25  
26     print("\nReporte de clasificación:")  
27     print(classification_report(y_true, y_pred, digits=4, zero_division=0))  
28  
29     return metrics  
30  
✓ <1 sec
```

Figura 49. Función de validación del modelo.

Y observamos los siguientes resultados en donde tiene un buen puntaje de roc_auc (métrica para clases desbalanceadas).

```

1
2  # =====
3  # 11. Evaluación en train, val y test
4  # =====
5  train_metrics = evaluate_model(best_model, X_train, y_train, "train")
6  val_metrics = evaluate_model(best_model, X_val, y_val, "validation")
7  test_metrics = evaluate_model(best_model, X_test, y_test, "test")
8
✓ 7 sec

```

===== TRAIN =====

accuracy: 0.9370
precision: 0.8220
recall: 0.9179
f1: 0.8673
roc_auc: 0.9845

Matriz de confusión:

```
[[62388  3801]
 [ 1571 17555]]
```

Reporte de clasificación:

	precision	recall	f1-score	support
0	0.9754	0.9426	0.9587	66189
1	0.8220	0.9179	0.8673	19126
accuracy			0.9370	85315
macro avg	0.8987	0.9302	0.9130	85315
weighted avg	0.9410	0.9370	0.9382	85315

Figura 50. Validación de “Train”

```

===== VALIDATION =====
accuracy: 0.8392
precision: 0.6364
recall: 0.6600
f1: 0.6480
roc_auc: 0.8809

Matriz de confusión:
[[19659 2404]
 [ 2168 4208]]

Reporte de clasificación:

```

	precision	recall	f1-score	support
0	0.9007	0.8910	0.8958	22063
1	0.6364	0.6600	0.6480	6376
accuracy			0.8392	28439
macro avg	0.7685	0.7755	0.7719	28439
weighted avg	0.8414	0.8392	0.8403	28439

Figura 51. Validación de “Val”.

```

===== TEST =====
accuracy: 0.8445
precision: 0.6476
recall: 0.6718
f1: 0.6595
roc_auc: 0.8869

Matriz de confusión:
[[19733 2331]
 [ 2092 4283]]

Reporte de clasificación:

```

	precision	recall	f1-score	support
0	0.9041	0.8944	0.8992	22064
1	0.6476	0.6718	0.6595	6375
accuracy			0.8445	28439
macro avg	0.7759	0.7831	0.7794	28439
weighted avg	0.8466	0.8445	0.8455	28439

Figura 52. Validación de “Test”.

A continuación revisaremos que nuestro modelo no está sobreentrenado.

```
1
2 # =====
3 # 12. Revisión básica de sobreajuste
4 # =====
5 print("\n==== REVISIÓN DE SOBREAJUSTE =====")
6 print(f"F1 Train: {train_metrics['f1']:.4f}")
7 print(f"F1 Validation: {val_metrics['f1']:.4f}")
8 print(f"F1 Test: {test_metrics['f1']:.4f}")
9
10 gap_train_val = train_metrics["f1"] - val_metrics["f1"]
11 gap_train_test = train_metrics["f1"] - test_metrics["f1"]
12
13 print(f"Gap Train-Val: {gap_train_val:.4f}")
14 print(f"Gap Train-Test: {gap_train_test:.4f}")
15
16 if gap_train_val > 0.05 or gap_train_test > 0.05:
17     print("Posible sobreajuste: el modelo rinde bastante mejor en entrenamiento que en validación/test.")
18 else:
19     print("No hay señales fuertes de sobreajuste según el gap de F1.")
20
21 # También revisar CV
22 cv_results = pd.DataFrame(search.cv_results_)
23 best_idx = search.best_index_
24
25 mean_train_cv = cv_results.loc[best_idx, "mean_train_score"]
26 mean_valid_cv = cv_results.loc[best_idx, "mean_test_score"]
27 std_valid_cv = cv_results.loc[best_idx, "std_test_score"]
28
29 print(f"\nMean train score CV: {mean_train_cv:.4f}")
30 print(f"Mean validation score CV: {mean_valid_cv:.4f}")
31 print(f"Std validation score CV: {std_valid_cv:.4f}")
32
✓ <1 sec
```

Figura 53. Prevención de “Overfitting”.

Y como era de esperar el modelo presenta indicios de sobreajuste sobre los hiperparametros por lo que hay que buscar mejores valores para que se desempeñe mejor en la generalización.

```
==== REVISIÓN DE SOBREAJUSTE ====
F1 Train: 0.8673
F1 Validation: 0.6480
F1 Test: 0.6595
Gap Train-Val: 0.2193
Gap Train-Test: 0.2078
Posible sobreajuste: el modelo rinde bastante mejor en entrenamiento que en validación/test.

Mean train score CV: 0.8678
Mean validation score CV: 0.6541
Std validation score CV: 0.0021
```

Figura 54. Resultados de sobreajuste.

Y con eso vemos la comparativa entre los distintos conjuntos de entrenamiento, validación y test.

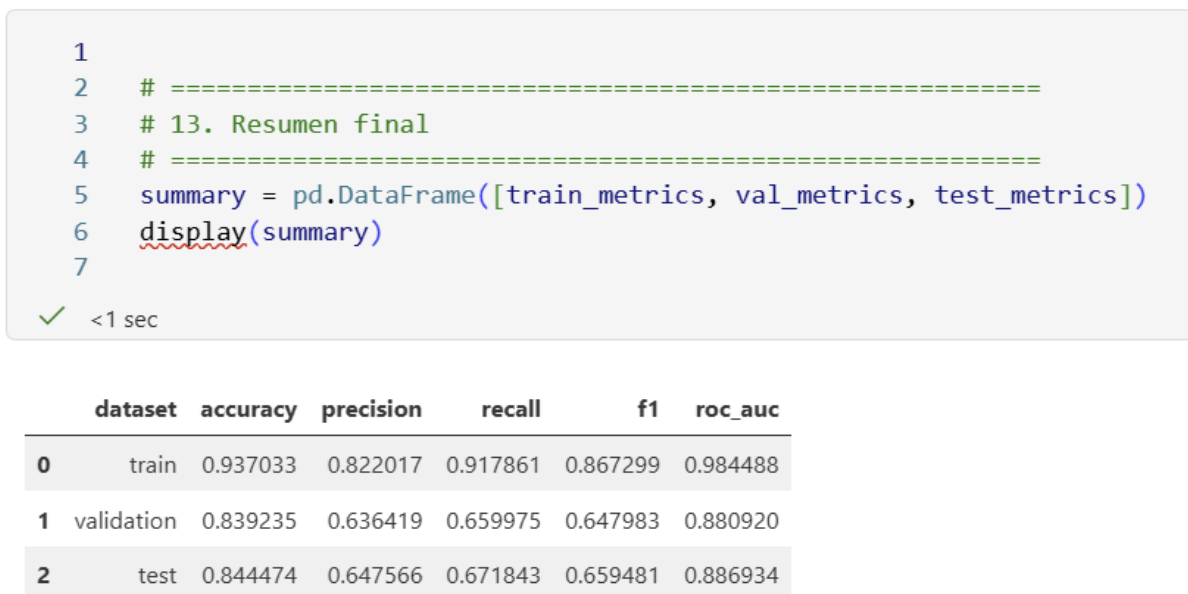
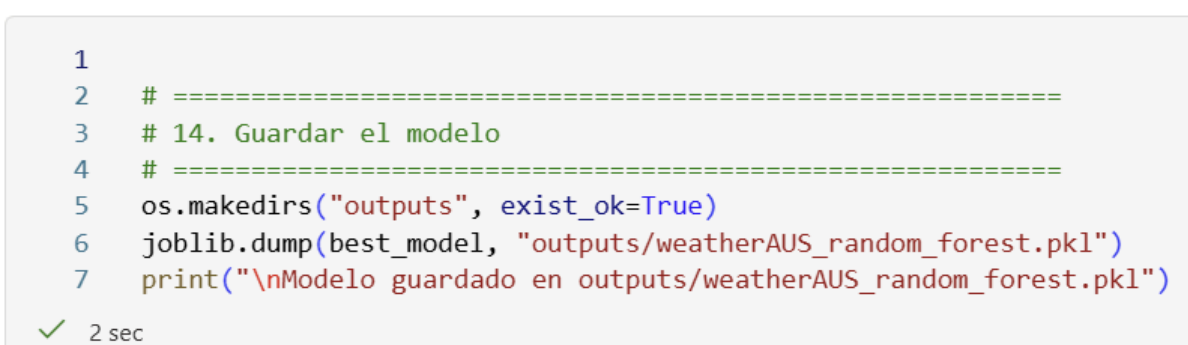


Figura 55. Métricas generales de las pruebas de entrenamiento.

Y por último, si nuestro modelo es relevante, lo guardaremos para ser utilizado en el futuro o ajustar sus pesos sobre datos nuevos sin tener que entrenar desde cero.



Modelo guardado en outputs/weatherAUS_random_forest.pkl

Figura 56. Exportar modelo.

Conclusión

El uso de plataformas de cómputo en la nube como Azure ML permite desarrollar soluciones de Machine Learning de manera eficiente, accesible y escalable. A través de esta práctica, se logró implementar un flujo completo que incluye la carga de datos, el entrenamiento de modelos y la obtención de resultados.

Asimismo, el uso de herramientas como AutoML demuestra cómo la automatización puede facilitar el desarrollo de modelos incluso para usuarios con menor experiencia.

En esta actividad logramos entrenar un modelo de “machine learning” desde cero haciendo uso de código convencional así como el producto de cero código que proporciona AutoML, el cual fue de gran utilidad para manejar un modelo sin necesidad de conocimientos avanzados de machine learning para llegar a un buen resultado de forma mucho más rápida para llegar a un modelo funcional. Adicionalmente cuenta con muchas más herramientas de administración de modelos y visualización rápida de métricas con las que no contamos con el código sencillo si no las implementamos.

Con todo esto, se reafirma la importancia de las plataformas en la nube como herramientas clave para el análisis de datos y la toma de decisiones en distintos ámbitos en entorno de desarrollo ágiles y propicios a la experimentación sin mucho gasto de recursos.

Bibliografía

Microsoft. (2024). Azure Machine Learning documentation. Recuperado de: <https://learn.microsoft.com/en-us/azure/machine-learning/>

Microsoft. (2024). What is Automated Machine Learning (AutoML)?. Recuperado de: <https://learn.microsoft.com/en-us/azure/machine-learning/concept-automated-ml>

Google. (2024). Google Cloud Machine Learning overview. Recuperado de: <https://cloud.google.com/learn/what-is-machine-learning>

Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow. (2019). O'Reilly Media.

Python for Data Analysis. (2017). O'Reilly Media.

Scikit-learn. (2024). Documentation. Recuperado de:
<https://scikit-learn.org/stable/documentation.html>